

Year Up United

Data Analyst Training Academy:

Data Visualization with Power BI

Week 10 Lab Workbook

This courseware is to be distributed to accompany instructor led training.

1.D.24 Table of Contents

MODULE 1 POWER QUERY FORMULA LANGUAGE (M).....

3

EXERCISE 1.A CONTOSO – PART 1, SHAPING FLAT FILE DATA WITH M.....

4

EXERCISE 1.B CONTOSO – PART 2, BUILDING DIMENSIONS

7

MODULE 2 WORKING WITH DAX

13

EXERCISE 2.A EXPLORING DAX FUNCTION REFERENCE

14

EXERCISE 2.B NORTHWIND – WORKING WITH PERCENTAGES

15

EXERCISE 2.C NORTHWIND – CONTEXT-BASED AGE CALCULATIONS.....

18

EXERCISE 2.D NORTHWIND – FLATTENING MANY-TO-MANY RELATIONSHIPS

20

MODULE 3 WORKING WITH VISUALIZATIONS.....

22

EXERCISE 3.A STARTING FROM “RETAIL ANALYSIS SAMPLE” REPORT

23

EXERCISE 3.B RETAIL ANALYSIS – GROUPING & CLUSTERING

29

EXERCISE 3.C RETAIL ANALYSIS – DRILL-THROUGHS & TOOLTIPS.....

33

EXERCISE 3.D CONTOSO – PART 3, ADDING VISUALIZATIONS

37

Module 1

Power Query Formula Language (M)

Exercise 1.A Contoso – Part 1, Shaping flat file data with M

Working with Contoso sample data

In this project we will be creating a report for the Contoso business that allows the user to compare online sales versus retail store sales. Unfortunately for us, you cannot access the underlying database but are limited to two files provided as flat exports of the data:

- An Excel spreadsheet containing summaries of Online Sales
- A tab delimited text file export of Retail Sales (over 160MB of text! Who made this?!)

Ultimately, we would like to create a report that lets us see retail and online sales by date, side-by-side, filterable by product category.

We will be faced with a few modelling challenges along the way. Whenever possible we will create Power Queries using the GUI ... but you'll see that we'll have to resort to writing some custom M code along the way.

Let's get started!

1. First download the two Contoso files from the course material into your **DataAnalytics > week-10** folder.
2. Next, open up a new blank Power BI project.
3. Give the project a name by saving it. Select **Save As** from the file menu and save the file to week-10 as **Contoso_M_Step_00.pbix**
4. Use the Home tab of the ribbon to **Get Data** from Excel, and load the file from the course material named **ContosoOnlineSalesReport.xlsx**
 - * Select the Excel sheet named **ContosoOnlineSales** and click load.
5. Click on **Get Data** again, this time choosing Text/CSV.
 - * In the "File Open" dialog window, change the file type drop-down (to the right of the "File name:" blank) to **All Files (*.*)** then select the file from the course material named **ContosoRetailSales.rpt**

* **Click Load.**

6. In Power BI, navigate to the **Table view** and take a moment to explore the two tables of data that you'll be working with.

* Notice that in **ContosoOnlineSales**, the time granularity of an entry is per **Month**.

* Notice that in **ContosoRetailSales**, the time granularity of an entry is per **Day**.

* Notice that in **ContosoOnlineSales**, the location granularity of an entry is per **State**.

* Notice that in **ContosoRetailSales**, the time granularity of an entry is per **City**.

7. Open Power Query Editor by clicking on Transform Data in the Home ribbon.

8. Change the data types of each existing column based on the following table; if prompted replace the current step instead of creating a new step:

Table	Column	Datatype
ContosoOnlineSales	Total Sales	Fixed Decimal Number
ContosoOnlineSales	State	Text
ContosoOnlineSales	Product	Text
ContosoOnlineSales	Category	Text
ContosoOnlineSales	CalendarMonth	Text
ContosoRetailSales	Location	Text
ContosoRetailSales	ProductName	Text
ContosoRetailSales	ProductCategoryName	Text
ContosoRetailSales	FullDateLabel	Date
ContosoRetailSales	TotalCost	Fixed Decimal Number

9. The Location column of the RetailSalesOrigin table contains both a City and State. Split this into two columns:

* Select the **ContosoRetailSales** table.

* Right click on the **Location** column and select **Split Column -> By Delimiter**.

* On the **Split Column by Delimiter** dialog, confirm that **Comma** is selected as the delimiter and click **OK**.

* It is hard to tell but new the **Location.2** column has a space at the beginning of each value. Right Click on the **Location.2** column and choose **Transform -> Trim**. (Alternatively, you could edit the previous step to include the space after the comma delimiter in the M expression used to split the column.)

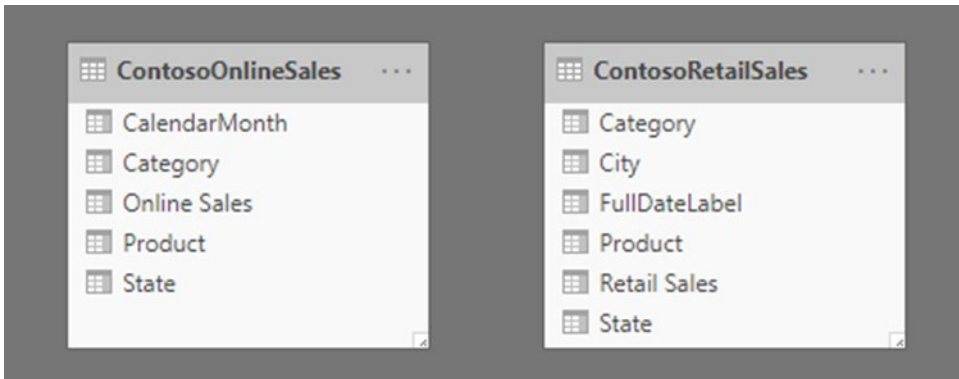
10. Rename the following columns:

	Old Name	New Name
ContosoRetailSales	Location.1	City
ContosoRetailSales	Location.2	State
ContosoRetailSales	ProductName	Product
ContosoRetailSales	ProductCategoryName	Category
ContosoRetailSales	TotalCost	Retail Sales
ContosoOnlineSales	Total Sales	Online Sales

- 11. Close the Power Query Editor window using Close & Apply. At this point Power BI will take a moment as the queries run against the data sources.**
- 12. Save the updated project as Contoso_M_Step_01.pbix**

Exercise 1.B Contoso – Part 2, Building Dimensions

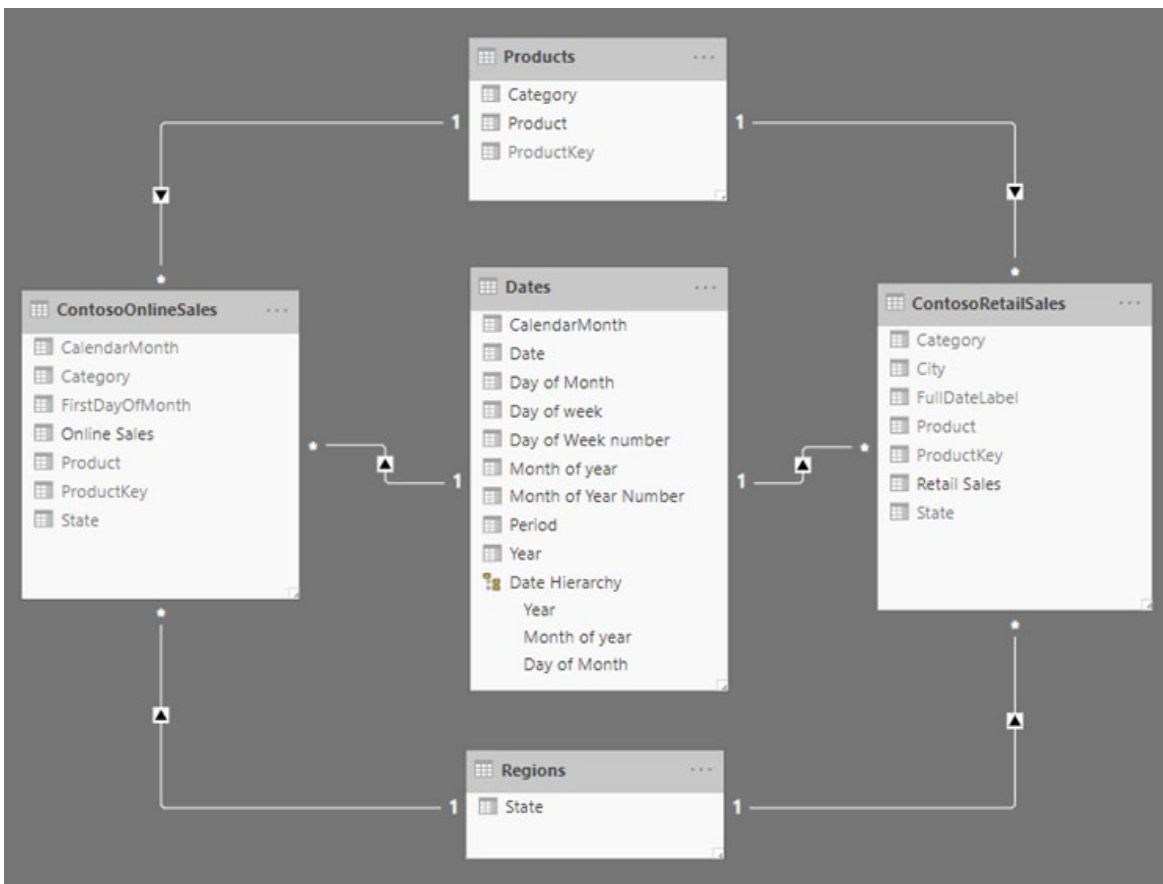
If you look at the current data model in Power BI, you will see it looks like this:



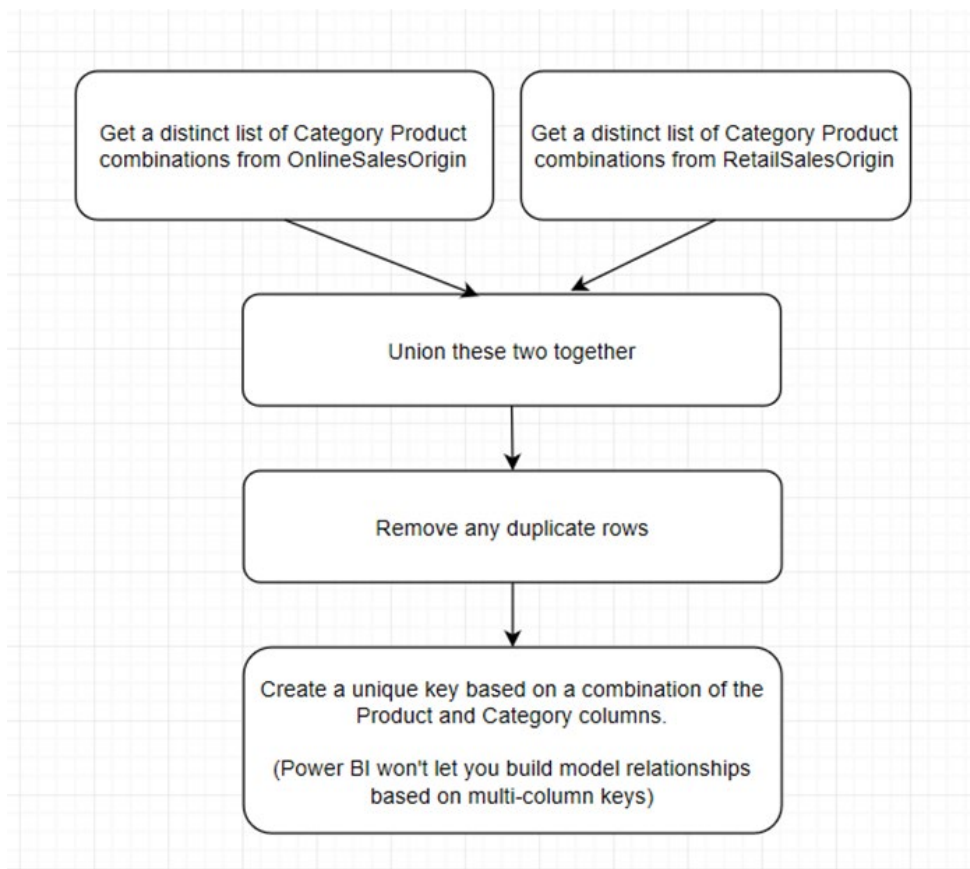
There are some problems to resolve when modeling this data:

- Online Sales are granular to the month, but Retail Sales are granular to the day.
- Nothing tells the model that a Product in OnlineSalesOrigin is the same as a Product in RetailSalesOrigin. The same is true for Category and State columns.

Here is the ideal model you want to reshape the data to support. This represents a star schema with two fact tables and three dimension tables:



Getting there is going to take a little work! As you're learning a new language like **M**, it's worth doing a little bit of planning before you dive in. Sketching out the steps involved on scratch paper (or a draw.io diagram) is a great first step:



Creating the Products table

1. Continue working with the previous file, and open the Power Query Editor.
2. Create a **New Source** based on a blank query.
3. Rename the new query from **Query1** to **Products**.
4. Right-click on the **Products** query and choose **Advanced Editor**.
5. Take a moment to read through the below M query to get a sense of what it is doing!
 - * If you need to, google “**Table.Combine**”, “**Table.Distinct**”, and “**Table.AddColumn**” and pull up the **Power Query Reference** pages on these functions.

- * When you are finished understanding what the code will do, type it out into the editor. (Yes, you must type it out the long way, not copy-paste.)

```
let
    UnionedProductData = Table.Combine(
        {ContosoOnlineSales, ContosoRetailSales},
        {"Category", "Product"}
    ),
    DistinctRows = Table.Distinct(UnionedProductData),
    AddKeyColumn = Table.AddColumn(
        DistinctRows,
        "ProductKey",
        each [Category] & "|" & [Product]
    )
in
    AddKeyColumn
```

- * Check your understanding: Looking at the above code, what are the three step names? What's happening in each step?

6. Add a new column named **ProductKey** to the **ContosoRetailSales** query by using the Advanced Editor to add to the let statement:

- * Make sure the circled statements match!

ContosoRetailSales

Display Options ▾ ?

```
let
    Source = Csv.Document(File.Contents("C:\HOTT-DAX-M\StarterFiles\ContosoRetailSalesForHOTT.rpt"),[Delimiter="
", Columns=5, Encoding=65001, QuoteStyle=QuoteStyle.None]),
    #"Promoted Headers" = Table.PromoteHeaders(Source, [PromoteAllScalars=true]),
    #"Changed Type" = Table.TransformColumnTypes(#"Promoted Headers",{{"Location", type text}, {"ProductName", type
text}, {"ProductCategoryName", type text}, {"FullDateLabel", type date}, {"TotalCost", Currency.Type}}),
    #"Split Column by Delimiter" = Table.SplitColumn(#"Changed Type", "Location", Splitter.SplitTextByDelimiter(", ",
QuoteStyle.Csv), {"Location.1", "Location.2"}),
    #"Changed Type1" = Table.TransformColumnTypes(#"Split Column by Delimiter",{{"Location.1", type text},
{"Location.2", type text}}),
    #"Renamed Columns" = Table.RenameColumns(#"Changed Type1",{{"Location.1", "City"}, {"Location.2", "State"},
{"ProductName", "Product"}, {"ProductCategoryName", "Category"}, {"TotalCost", "Retail Sales"}}),
    ,
    #"AddProductKey" = Table.AddColumn(#"Renamed Columns", "ProductKey", each [Category] & "|" & [Product])
in
    #"AddProductKey"
```

Inserted step text (make sure to include the comma before the new step):

```
,
#"AddProductKey" = Table.AddColumn(#"Renamed Columns", "ProductKey",
each [Category] & "|" & [Product])
```

7. Also add a **ProductKey** column to the **ContosoOnlineSales** query, using the same pattern that you applied on the above step.
8. Add another column to the **ContosoOnlineSales** table, but this time utilize the dialog found at **Power Query Editor > Add Column Ribbon > Custom Column button**. Use the following values:

Custom Column

Add a column that is computed from the other columns.

New column name
FirstDayOfMonth

Custom column formula ⓘ
= #date({
 Number.FromText(Text.Middle([CalendarMonth],0,4)),
 Number.FromText(Text.Middle([CalendarMonth],4,2)),
 1
})

Available columns
Online Sales
State
Product
Category
CalendarMonth
ProductKey

<< Insert

[Learn about Power BI Desktop formulas](#)

✓ No syntax errors have been detected.

OK Cancel

9. Change the datatype of the **FirstDayOfMonth** column to Date.
10. Create another new query, and name it **Regions**.
11. Using the advanced editor, enter the following query:

```
let
    UnionedProductData = Table.Combine(
        {ContosoOnlineSales, ContosoRetailSales},
        {"State"}
    ),
    DistinctRows = Table.Distinct(UnionedProductData)
in
    DistinctRows
```

12. Create another new query, and name it **Dates**.
13. Using the advanced editor, enter the following query. NOTE: For this one, you can cut and paste from the file included in the material, **M_DateTableExpression.txt**

```

1  let
2      StartDate = #date(2000,1,1),
3      EndDate = #date(2020,1,1),
4      NumberOfDays = Duration.Days(Duration.From(EndDate-StartDate)),
5      Source = List.Dates(StartDate,NumberOfDays,#duration(1,0,0,0)),
6      TableFromList = Table.FromList(Source,Splitter.SplitByNothing(),{"Date"}),
7      ChangeType = Table.TransformColumnTypes(TableFromList,{"Date", type date}),
8      Year = Table.AddColumn(ChangeType,"Year",each Date.Year([Date]),type number),
9      MonthOfYearNumber = Table.AddColumn(Year,"Month of Year Number",each Date.Month([Date]),type number),
10     MonthOfYearName = Table.AddColumn(MonthOfYearNumber,"Month of year",each Date.ToText([Date],"MMMM"),type text),
11     DayOfMonth = Table.AddColumn(MonthOfYearName,"Day of Month",each Date.Day([Date]),type number),
12     DayOfWeekNumber = Table.AddColumn(DayOfMonth,"Day of Week number",each Date.DayOfWeek([Date]),type number),
13     DayOfWeekName = Table.AddColumn(DayOfWeekNumber,"Day of week",each Date.ToText([Date],"dddd"),type text),
14     PeriodNumber = Table.AddColumn(DayOfWeekName,"CalendarMonth",each Date.ToText([Date],"yyyyMM"),type text),
15     Period = Table.AddColumn(PeriodNumber,"Period",each Date.ToText([Date],"MMM") & "-" & Date.ToText([Date],"yy"),type text)
16 in
17     Period

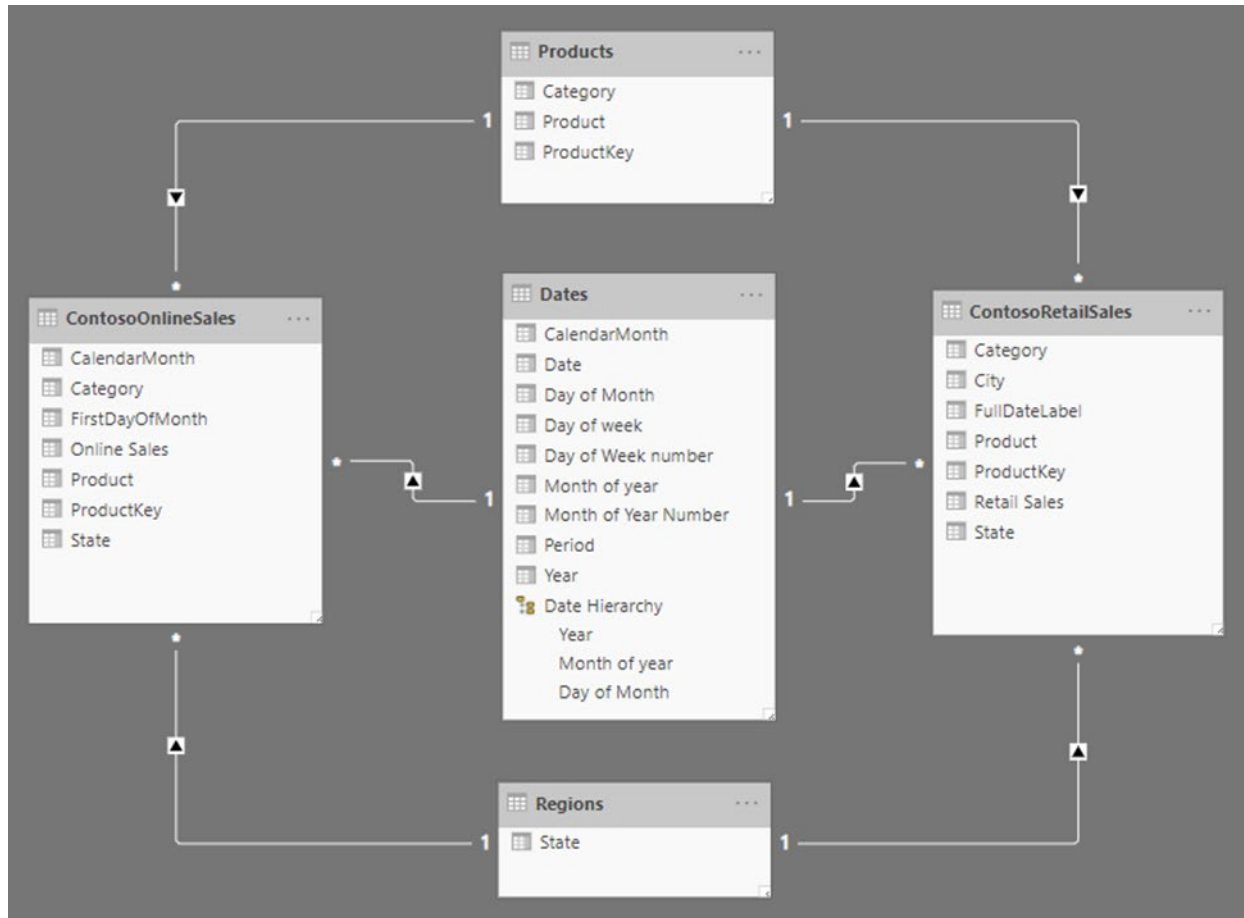
```

14. Close the Power Query Editor using Close & Apply.
15. Back in Power BI Desktop, navigate to the Model view.
16. View the **Dates** table.
17. Configure the **Month of Year** column to order by the **Month of Year Number** column.
18. Configure the **Day of Week** column to order by the **Day of Week Number** column.
19. Configure the **Period** column to order by the **CalendarMonth** column.
20. Create a hierarchy in the Date table named **Date Hierarchy** with the following columns:
 - * Year
 - * Month of year
 - * Day of Month
21. Save the project as **Contoso_M_Step_02.pbix**

Shaping the Contoso Semantic Model

22. Using the **Model** view, build out the following relationships:

- * Hint: The **ContosoOnlineSales** table does not have a specific date, so you will need to build the relationship on the **FirstDayOfMonth** date column that we had calculated out



23. Hide every column in the **ContosoOnlineSales** table except for **Online Sales**

24. Hide every column in the **ContosoRetailSales** table except for **Retail Sales**

25. Hide the **ProductKey** column in the **Products** table.

26. Save the project as **Contoso_M_Step_03.pbix**

Module 2

Working with DAX

Exercise 2.A Exploring DAX Function Reference

Introduction

Once again, getting an idea of the kinds of functions in DAX may help you later when you are brainstorming for useful calculations or working through a specific problem.

1. Spend 10 minutes reviewing the **DAX function reference** document available online from Microsoft Learn, including a brief look at the categories of available functions. Also take note of where to find the search box to quickly locate information on a specific function:

<https://docs.microsoft.com/en-us/dax/dax-function-reference>

2. Some particular ideas for exploration:
 - * What kind of aggregation functions have you used in SQL or Python? What equivalent functions appear in the DAX library?
 - * Similarly, looking at the available text functions, what would you equate these to in SQL or Python?
 - * What kinds of things can you do with “information functions”? How about “logical functions”?

No written responses are required for the above questions, but do take your time with exploring this resource!

Exercise 2.B Northwind – Working with Percentages

Introduction

In this exercise we will demonstrate how percentages can be a dangerous value when expressed as a column instead of a measure.

1. In your Northwind project, in the **Table view**, select the **Discount Percent** column of the **Sales** table.
2. Using the **Column tools** tab of the ribbon, set the format of **Discount Percent** to **Percentage**.
3. Observe the data in the **Discount Percent** column. You may need to scroll down a while in the data before you see numbers other than **0 . 00%**.
4. In the **Report view**, create a new report tab named **Percentage Problems**.
5. Add a table with the following fields:
 - * Category
 - * Sales Amount
 - * Discount Amount
 - * Discount Percent
6. If you look at the numbers in the **Discount Percent** column you will notice that the **Discount Percent** for the **Beverages** category is 2,500%. It should be obvious that this number is not useful to us. By default, all the percentages have been added together. An implicit measure was created that uses the expression `SUM([Discount Percent])`.
7. Using the steps outlined on the next page, change the implicit measure of the **Discount Percent** column to use an average instead of a sum:

The screenshot shows the Power BI Desktop interface. On the left, a table displays sales data categorized by product type. The 'Discount Percent' column shows values ranging from 6.19% to 12.74%. On the right, the 'Visualizations' pane is open, showing the 'Columns' section with 'Category', 'Sales Amount', 'Discount Amount', and 'Discount Percent'. A context menu is open for 'Discount Percent', showing options like 'Sum', 'Average', 'Minimum', etc. The 'Average' option is highlighted with a blue circle and the number 3.

Category	Sales Amount	Discount Amount	Discount Percent
Beverages	\$267,868.18	\$18,658.77	6.19%
Condiments	\$106,047.09	\$7,647.67	7.19%
Confections	\$167,357.23	\$9,741.88	5.82%
Dairy Products	\$234,507.29	\$16,823.22	7.19%
Grains/Cereals	\$95,744.59	\$4,982.21	5.20%
Meat/Poultry	\$163,022.36	\$15,166.44	9.29%
Produce	\$99,984.58	\$5,284.02	5.28%
Seafood	\$131,261.74	\$10,361.35	7.89%
Total	\$1,265,793.04	\$88,665.55	6.99%

- * Select the Table
 - * Click on the **Chevron** next to the **Discount Percent** column in the **Columns** section of the **Visualizations** pane
 - * In the context menu that appears, change the selected item from **Sum** to **Average**
8. Observe the values in the **Discount Percent** column. You will find them to be in the 4-6% range, and this looks like it could be correct. However, this figure represents an average of the values at the detail level, and that is not the **Discount Percent** when summarizing detail at the **Category** level.
- * This number is not useful and is dangerous because it could be easily misinterpreted!
 - * Looking at the **Beverages** row, using a calculator you'll find that $\$18,658 / \$267,868 = 0.06965$. The value listed here is 6.19%.
 - * To solve this problem, we need to use a measure. **Measures** are calculated separately at each grouping level as needed by the report where it is utilized. They also consider implicit filters based on rows the user may have selected within the report.

9. In the **Table view**, right-click on the name of the **Sales** table and choose **New Measure**.
10. Enter the following in the formula bar:
$$\text{Discount Percent Measure} = \text{SUM}(\text{Sales}[\text{Discount Amount}]) / \text{SUM}(\text{Sales}[\text{Sales Amount}])$$

You will notice that after adding the measure, a column does not appear in the table. Measures are not actually calculated until used in a report.
11. To protect against the **[Discount Percent]** column from being used in reports, mark it as hidden from the report view. This will teach users to use the **[Discount Percent Measure]** instead.
12. Navigate to the Report view, and add the **[Discount Percent Measure]** to the table.
13. The **[Discount Percent Measure]** column should display as a percentage.
 - * If it does not, update the measure's format in the **Table view** using the **Measure tools** tab of the ribbon.
14. In the table, observe the difference between the **Discount Percent Measure** and **Discount Percent** columns.
15. Remove the **[Discount Percent]** column.
16. Save your work as: **NorthwindModeling_Step10.pbix**

Exercise 2.C Northwind – Context-based Age Calculations

Introduction

Previously we added a column to the **Employees** table that calculates the age of the employee. What if we want to look at the ages of employees at the time, they made specific sales? This age would belong to the Sale record, instead of the Employee record.

1. In your **Northwind Modeling** project, create a new **Report** tab named **Ages**.
2. Create a clustered column chart with **Employee 's Age** as the X-axis and **Sales Quantity** as Y-axis.
3. While there is nothing obviously wrong with these results, notice that this dataset is not updated through the current year. Therefore this calculation shows us the current age of the employees, which is different than their age at the time they were selling products! That could be potentially misleading.
4. Notice that the number of distinct ages found is 8 (count the bars).
5. Create a new **Calculated Column** on the **Sales** table using the following expression:
`Emp Age At Order = FLOOR( (RELATED(Order[Order Date]) - RELATED(Employee[Employee Birth Date])) / 365,1)`
6. Back on the **Ages** report page, replace the **Employee's Age** with the **Emp Age At Order** as the axis field on the chart.
7. Notice that we have over 20 distinct ages now, as employees changed age over the three-year period that sales were recorded.
8. Add a **Table** visualization to the report with the following columns:
 - * `Orders[OrderDate].[Date Hierarchy].[Year]`
 - * `Sales[Quantity]`
 - * `Sales[Emp Age At Order]`
 - * `Employee[Employee's Age]`

9. Change the implicit measure that was created for both **Sales [Emp Age At Order]** and **Employee [Employee 's Age]** to display average instead of sum.
10. Notice that the **Average of Emp Age at Order** values are different for each year, while the **Average of Employee's Age** are the same. This is because **Emp Age at Order** belongs to the **Sales** table, so it is averaging one age per order instead of one age per employee.
11. Save your work as: **NorthwindModeling_Step11.pbix**

Exercise 2.D Northwind – Flattening Many-to-Many Relationships

Introduction

One thing that Power BI is not great with is dealing with many-to-many relationships between dimensions. In the Northwind database we have Employees that can belong to multiple territories, and multiple Territories can be managed by the same Employee. To make reporting simpler, we're going to collapse this descriptive list into a single column of text.

1. Open up the **Northwind Modeling** project that we have been building throughout the course.
2. Add a **Calculated Column** to the **EmployeeTerritories** table named `[Employee Territory]`. Set it equal to the **Territory** column from the related **Territories** row.
3. Add a **Calculated Column** to the **EmployeeTerritories** table named `[Employee Region]`. Set it equal to the **Region** column from the related **Region** row.
4. Hide the **Territories** and **Region** tables from the report view.
5. Add a **Calculated Column** to the **EmployeeTerritories** table named `[Employee Territory Description]`, combine the **Territory** and **Region** fields together to get an output that looks like this (include the braces):

`[Wilton (Eastern)]`
`[Atlanta (Southern)]`
6. Add a **Calculated Column** to the **Employees** table named `[Employee Territories]` that uses the `CONCATENATEX()` iterator to create a string that contains all the territories that an employee covers, delimited with a comma and a space. Set it up so that you get values that look like this:

```
[Phoenix (Western)], [Scottsdale (Western)], [Bellevue (Western)],  
[Redmond (Western)], [Seattle (Western)]
```

* Hint: You will need to use both `CONCATENATEX()` and `RELATEDTABLE()`

7. Create a new **Report** page named **Employees**. Add a table that lists employees and the territories they belong to as well as the total sales amount for each employee.
8. Notice the horizontal scrolling created by the long text strings in the **Employee Territories** column. Experiment with resizing the table and resizing the column width to best display the data.
9. Save your work as: **NorthwindModeling_Step12.pbix**

Module 3

Working with Visualizations

Exercise 3.A Starting from “Retail Analysis Sample” Report

Introduction

For this exercise, we will be using the Retail Analysis Sample Power BI report, included in the course materials (and also available from Microsoft Learn).

Exploring Power BI Visualizations

1. Open the sample report **RetailAnalysisSample.pbix**
2. Go to the **Overview** page of the report.
3. Review each of the visuals and consider what each represents.
 - * What types of visuals are included on this page?
 - * What is being displayed in each?
4. On the pie chart, click on **Lindseys** to filter the report page.
 - * How many stores does the Lindseys chain include? In how many districts?
 - * Now click on Fashions Direct to change the filter. How many stores does this chain include? In how many districts?
5. On the map, click on **New Store** in the legend to re-filter the report page to data relating to new stores only. Then hold down Ctrl and click again on **Lindseys** in the pie chart to add this as a second report filter.
 - * How many new stores belong to the Lindseys chain?
 - * What is the “This Year Sales” amount so far for new Lindseys stores? (Hint: Remember that you can hover to see details in the tooltip.)
6. On the column chart, click on **Andrew Ma** in the legend to re-filter the report page.
 - * What district does this person manage? How many stores does it include? How many of these are new stores?

- * Notice that the map and the card visual do not agree. Which do you think is reliable here?
7. Click in the background of any visual to remove filters and return to the original view.
 8. Imagine your boss has sent this file to you with the following note: “Can you please look at this and clean up the **Overview** page to make it more readable? We want to be able to filter by district manager, but squinting at that column chart is giving me a headache. Also, that pie chart is boring. I hate pie charts. Can you make it look less boring?”
 - * What approaches might you take to improve this page based on your boss’s comments?
 - * What follow-up questions might you want to ask?
 9. Let’s say you decide to change the column chart from clustered columns to stacked columns. How would you do that? (Hint: It should take you only one click in the Visualizations pane.)
 10. Similarly, change the format of the pie chart to a donut chart.
 11. What can you add to this report page to make filtering by district manager even easier?
 - * To create some available white space, first delete the card visual for New Stores, since we already noticed that wasn’t working quite as intended.
 - * In the **Visualizations** pane, click the button for **Slicer**. Then in the **Data** pane, expand the table for **District**, then select **DM** to add that field to the slicer.
 - * Resize the slicer so that all names are visible in the list. Reposition the slicer and the **Total Stores** card to be evenly spaced on the page.
 12. Spend some time exploring the filter interactions on your updated report page.
 - * What is the difference between selecting **Allan Guinot** from the slicer, versus clicking on **Allan Guinot** in the legend of the column chart, versus clicking on district **LI – 01** in the bubble chart?

- * Hold down Ctrl to multi-select **Andrew Ma, Carlos Grilo, Tina Lassila, and Valery Ushakov** in the slicer. Then in the map, click on **Same Store** to focus on sales performance of existing stores only. Next hold down Ctrl and click on **Feb** in the column chart to focus just on February sales. Within this view, where can you find the total dollar value of February sales for these four managers? How could you quickly find store-level totals?

13. Save the file to DataAnalytics > week-10 as **Retail_Analysis_Step_01.pbix**

Enhancing the Report: New Scenario

Your boss was pleased with the changes you made to the Overview page. Now they would like you to add another page that gives a more detailed comparison of the performance of each retail chain, Lindseys and Fashion Direct. This page along with the rest of the report will be used in a hybrid-format meeting your boss has coming up, where it will be displayed onscreen in a meeting room as well as screenshared.

14. In the report that you have been working with, create a new report page by clicking the plus tab at the bottom of the canvas. Rename this page to **Lindseys-FD Comparison**.
15. Go to one of the other report pages (**Overview, District Monthly Sales, or New Stores**), and copy the text box used for the page title. You can do this by clicking on the header of the text box visual to select it, then using Ctrl + C to copy.
16. Return to your new report page and paste the title text box, then update the text to read **Comparison: Lindseys & Fashions Direct**. Resize the text box if needed to fit the text all on one line, and make sure it is centered on the page by going to the **Format** tab of the ribbon, then clicking **Align > Align Center**.
17. Using the **Visualizations** pane, add a simple table that includes the following columns:
 - * **Chain** (found in the **Store** table)
 - * **TotalSalesLY** (found in the **Sales** table)
 - * **This Year Sales > Value** (found in the **Sales** table)

18. With the table still selected, switch to the format  view in the **Visualizations** pane. Expand the options for **Values**, and increase the font size to 14. Then resize the table visualization to fit the displayed data, and move it to the upper left of the canvas.
19. Add a line chart to the page that includes:
 - * Y-axis: **This Year Sales > Value** (from the **Sales** table)
 - * X-axis: **Month** (from the **Time** table)
 - * Legend: **Chain** (from the **Store** table)
20. With the line chart still selected, in the **Visualizations** pane update the x-axis field using the down-chevron menu to display the Month, rather than the Date Hierarchy.
 - * Why does Month make sense here? When would you want to use the Date Hierarchy instead?
21. With the line chart still selected, resize the chart horizontally so that the month labels are spaced out enough to be easy to read. Experiment with resizing the chart vertically as well.
 - * What is the visual impact of making the y-axis shorter versus longer (i.e. making the visual an elongated rectangle versus more of a square)? How might you use this to impact the story that your report page is telling?
 - * Resize the line chart to a square-ish shape and move it to the left of the canvas, underneath the table visual.
22. Add a donut chart visualization that includes the following:
 - * Legend: **Territory** (from the **Store** table)
 - * Values: **This Year Sales > Value** (from the **Sales** table)
23. Copy and duplicate the donut chart you just created, so that you have two side by side.
 - * Select the first donut chart.
 - * In the **Data** pane, find the field for **Chain** in the **Store** table.

- * Drag **Chain** into the **Filters** pane, and drop it under **Filters on this visual** into **Add data fields** here.
 - * Check the box under the new **Chain** filter for **Fashions Direct**.
 - * Select the second donut chart. Follow the same steps, but this time apply the filter for **Lindseys**.
24. Notice that the default titles applied to your two donut charts are the same. One at a time, select each chart, and use the format  view in the **Visualizations** pane to update the chart title. (Hint: you will find the **Title** field under the **General** tab of the format options.)
- * First chart: `Fashions Direct - This Year Sales by Territory`
 - * Second chart: `Lindseys - This Year Sales by Territory`
 - * For both charts, increase the font size of the chart title to 12 point, and change the text color to black. Update the title's alignment to be centered at the top of the chart.
 - * Why does it make sense to have two donut charts in this instance? What advantage does it present in the scenario here?
25. Notice that each donut chart accurately represents the relative size of sales territories by total dollar value within each chain. However, we can also see from this report page that overall sales for Fashions Direct are more than twice those of Lindseys. Resize the two donut charts to more proportionately represent each chain's sales relative to each other.
26. Add a map visualization to the page that includes:
- * Location: **PostalCode** (from **Store** table)
 - * Legend: **Chain** (from **Store** table)
 - * Bubble size: **This Year Sales > Value** (from **Sales** table)
27. Move the map to the lower right of the canvas, and experiment with adjusting its size and shape. Notice how the map re-zooms to fit the data, based on the dimensions of the chart.

- * Optional: You can use the format  options in the **Visualizations** pane to turn off **Auto zoom**, by toggling the switch under **Map Settings > Controls**.

28. Finally, add a matrix to the page that includes:

- * Rows: **Category** (from **Item** table)
- * Columns: **Chain** (from **Store** table)
- * Values: **Total Sales Var %** (from **Sales** table)

29. Use the format  options for the matrix to bump up the text size of the **Values** to be easier to read, e.g. 11 or 12 point font, while keeping the matrix sized to fully fit in the canvas without vertical or horizontal scrolling.

30. Save your updates as **Retail_Analysis_Step_02.pbix**

Exercise 3.B Retail Analysis – Grouping & Clustering

Introduction

In this exercise, we will continue making enhancements to the Retail Analysis report.

1. Open your Retail Analysis project (last saved as Retail_Analysis_Step_02.pbix)
2. Create a new report page by clicking the plus tab at the bottom of the canvas.
Rename this page to `Regional Manager Planning`.
3. Add a text box to create a page title with the text `Regional Manager Planning – Proposed Sales Regions`.
4. Add a clustered column chart at the upper left that includes:
 - * X-axis: **Month** (date hierarchy)
 - * Y-axis: **TotalSales**
 - * Legend: **District** (from the **District** table)
5. In the chart visual, click the forked downward arrow  twice to display columns at the month level.
6. In the Visualizations pane, click the down chevron next to **District** and choose **New group**.
7. In the **Groups** dialog, leave the default name as `District (groups)`. Using **Ctrl +** click to multiselect members of a group, create the following three groups, then click OK:
 - * `FD – 01 & FD – 02`
 - * `FD – 03 & FD – 04`
 - * `L1 – 01 & L1 – 02 & L1 – 03 & L1 – 04 & L1 – 05`
8. Notice that the chart's legend has been updated to reflect the three groups you just created as **District (groups)**, which is also now an available selection in the **Data**

pane within the **District** table. You can reuse this set of groups in other visualizations!

9. Add a map visualization to the lower left that includes:
 - * Location: **PostalCode** (from **Store**)
 - * Legend: **District (groups)** (from **District**)
10. Add a ribbon chart to the upper right that includes:
 - * X-axis: **Month** (again, expand down the date hierarchy to the month level)
 - * Y-axis: **This Year Sales > Value** (from **Sales**)
 - * Legend: **District (groups)** (from **District**)
11. Add a combined line and clustered column chart at the lower right that includes:
 - * X-axis: **Month** (again, expand down the date hierarchy to the month level)
 - * Column y-axis: **Total Units This Year** (from **Sales**)
 - * Line y-axis: **Gross Margin This Year** (from **Sales**)
 - * Column legend: **District (groups)** (from **District**)
12. For the combined chart, in the formatting options under **Lines > Shape**, change the **Line style** to dotted, and increase the **Stroke width** to 3 px.
13. Save your updates as **Retail_Analysis_Step_03.pbix**

Clustering

14. Continue working with your Retail Analysis report. Create a new report page by clicking the plus tab at the bottom of the canvas. Rename this page to **Sales Margin / Sq Ft Sales**.
15. Add a text box to create a page title with the text **Comparing Sales/Sq Ft and Gross Margin**.
16. Add a scatter chart at the upper left that includes:

- * Values: **StoreNumberName** (from **Store**)
 - * X-axis: **Sales Per Sq Ft** (from **Sales**)
 - * Y-axis: **Gross Margin This Year** (from **Sales**)
17. In the top right corner of the scatter chart visualization, click the ... icon and choose **Automatically find clusters**.
18. In the **Clusters** dialog, leave the default name as **StoreNumberName (clusters)**, and set the **Number of clusters** to 4. Then click OK.
19. After taking a moment to process, your scatter chart should be updated to show **StoreNumberName (clusters)** in the legend. This value is also now available in the **Store** table for use in other visualizations.
- * What do these clusters represent? Consider what is being compared on the x- and y-axis. Which clusters represent the most valuable stores?
20. Add a treemap visualization at the lower left that includes:
- * Category: **StoreNumberName (clusters)** (from **Store**)
 - * Details: **District** (from **District**)
 - * Values: **This Year Sales > Value** (from **Sales**)
21. Add a stacked bar chart, resized to fill the right side of the canvas, that includes:
- * Y-axis: **Month** (again, expand down the date hierarchy to the month level)
 - * X-axis: **This Year Sales > Value** (from **Sales**)
 - * Legend: **District (groups)** (from **District**)
 - * Small multiples: **StoreNumberName (clusters)** (from **Store**)
22. Spend a few moments considering what this stacked bar chart represents:
- * **District (groups)** is reused from the page created in the last exercise, Regional Manager Planning. This new chart compares those manually created group to the four generated clusters identified by Power BI.

- * How easy or difficult is it to read this chart? The legend is reusing the same color scheme from the Regional Manager Planning page, but here it appears ambiguous when placed alongside the same colors applied to the four clusters. You can clarify what each visualization represents by changing the colors used.
- * With the cluster chart selected, in the formatting options of the **Visualizations** pane, expand **Markers** and change the colors to shades of blue and gray.
- * Select the treemap chart and make the same color changes in **Colors**, consistent with what you selected for the scatter chart.
- * Spend another minute clicking on each value in the bar chart legend to filter the page by district group, and consider which clusters are most represented for each.

23. Save your updates as **Retail_Analysis_Step_04.pbix**

Exercise 3.C Retail Analysis – Drill-throughs & tooltips

Creating a Drill-through Page

In this exercise, we will continue making enhancements to the Retail Analysis report.

1. Open your Retail Analysis project (last saved as Retail_Analysis_Step_04.pbix)
2. Create a new report page by clicking the plus tab at the bottom of the canvas.
3. Rename this page to `Store Details`.
4. Right click the tab again and select **Hide Page**. This will be used as a drill-through page.
5. Add a text box to create a page title with the text `Total Sales by Store and by Category`.
6. Add a matrix that includes:
 - * Rows (include both):
 - DM** (from **Store**)
 - StoreNumberName** (from **Store**)
 - * Columns:
 - Category** (from **Item**)
 - * Values:
 - TotalSales** (from **Sales**)
7. In the top left of the matrix, click the forked downward arrow  once to display the store names underneath each district manager.
8. Resize your matrix as wide as necessary to eliminate horizontal scrolling. Also fill the canvas vertically, but notice that it is not possible to eliminate vertical scrolling due to the size of the matrix.

9. Click on the background canvas so that nothing is selected. In the Visualizations pane, drag **StoreNumberName** (from **Store**) into the space **Add drill-through fields here**.
10. Also add **Category** (from **Item**) as a drill-through field for the page.
11. Notice that adding drill-through fields has also added a back arrow  to the top left corner of the page.
12. You can also add drill-through fields to an existing page.
 - * Go to the Regional Manager Planning page that you previously created.
 - * Add **StoreNumberName** (from **Store**) as a drill-through field for the page.
13. Explore these drill-through interactions by going to the **District Monthly Sales** page of the report.
14. In the column chart **This Year Sales by StoreNumberName**, hover over a column and note the indication in the tooltip that drill-through is available.
15. Now right-click on the column and look at the available options for **Drill through**. What happens when you click on each?
16. In the scatter chart **Total Sales Variance %, Avg \$/Unit TY and This Year Sales by Category and Category**, right click on a bubble and look at the available options for **Drill through**. Why is the option here different? What do you learn by drilling through?
17. Return to the Store Details page.
18. How can you scrub the applied drill-through filters? Hint: Check the **Visualizations** pane, where you added the drill-through fields.
19. Do you notice anything interesting about the currently available drill-through fields? What changed?

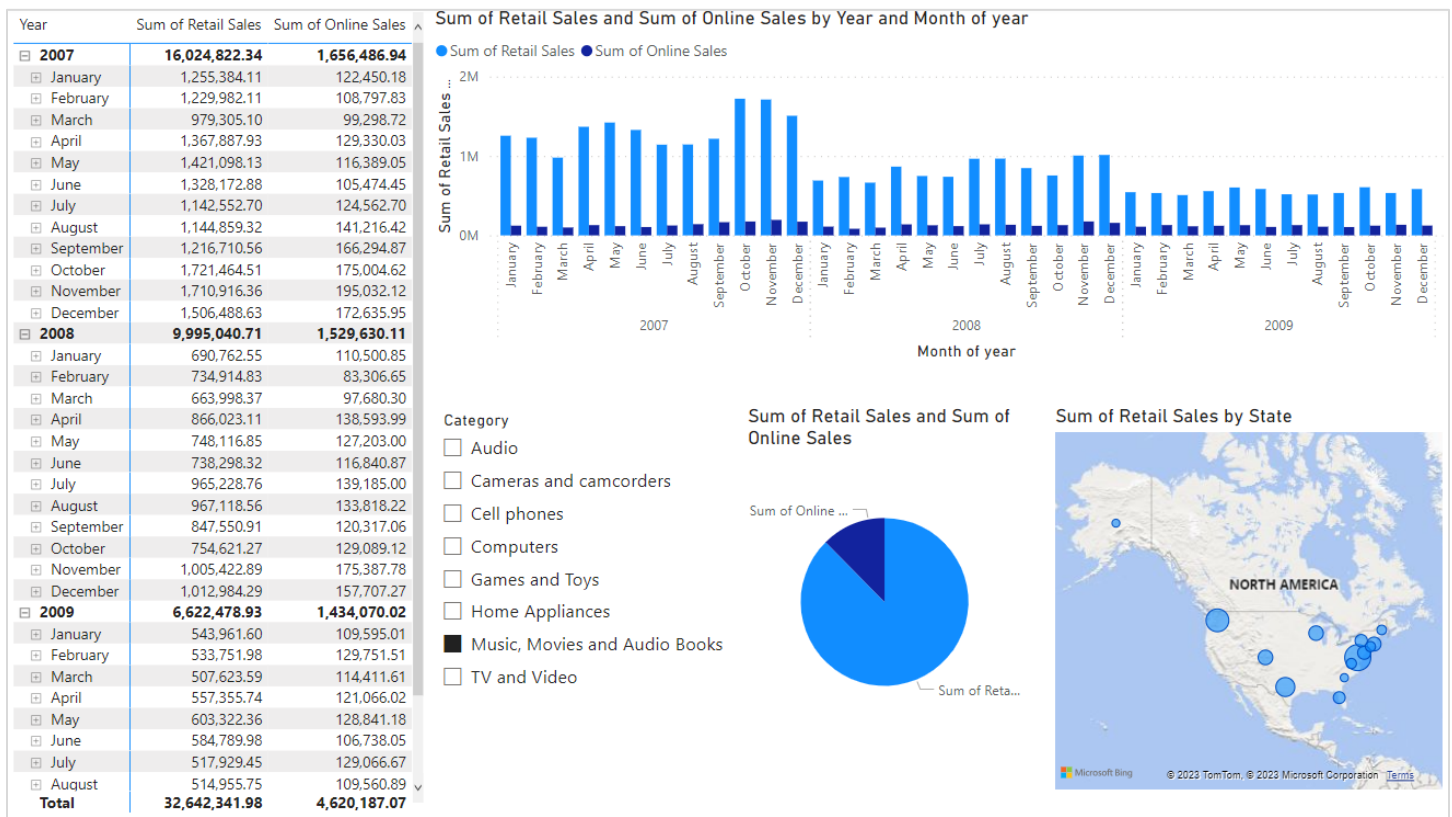
20. Return to the **Regional Manager Planning** page. Hover over a marker in the map visual to look at the information included in the tooltip that pops up. Not very illuminating, is it? Let's make a custom tooltip page to use instead.
21. Create a new page and name it Map Tooltip, then mark this page as hidden.
22. In the Visualizations pane, click into the formatting options. Under **Canvas settings**, change the **Type** to **Tooltip**. Confirm under **Page Information** that **Allow use as tooltip** is toggled to **On**.
23. Add a multi-row card visualization to the canvas. Include the following fields:
 - * **DM** (from **Store**)
 - * **Name** (from **Store**)
 - * **Total Sales** (from **Sales**)
 - * Is there another field you think would be interesting to use here? Add it!
24. Return to the **Regional Manager Planning** page. Click on the map visual to select it, then go to the formatting options in the **Visualizations** pane. Under **General**, expand the settings for **Tooltips**, and change **Page** to the page you just created, **Map Tooltip**.
25. Now hover over a marker on the map. Note that the tooltip uses the formatting of the page you created, and it is automatically filtered based on the marker selected.
26. To make the formatting of your custom tooltip match the other tooltips in the report, go back to the **Map Tooltip** page and make the following changes within the formatting options in the **Visualizations** pane:
27. With no visualization selected, under **Canvas background**:
 - * **Color**: Black, 20% lighter
 - * **Transparency**: 5%
28. With the card visualization selected, under **Callout values**:
 - * Increase font size to 12

- * Change font color to White, 10% darker
29. You can also play with the size of the canvas by changing the type to **Custom**, then entering a custom size such as **Height: 1500 px** and **Width: 250 px**
 30. Resize the card visualization so that all values display inline vertically, not side by side, and align the card to the center of the canvas. Try to get the vertical bar of the card visual to be centered vertically as well.
 31. When you are done formatting your tooltip page, return to the **Regional Manager Planning** page to look at the changes.
 32. Save your updates as **Retail_Analysis_Step_05.pbix**

Exercise 3.D Contoso – Part 3, Adding Visualizations

Visualizing Contoso Data

1. Open your Contoso project (last saved as **Contoso_M_Step_03.pbix**)
2. Using the skills you have developed in this class, create a report that looks like this:



3. Save your updated project as **Contoso_M_Step_04.pbix**

End of Week 10 Lab Workbook